Umais Ur Rehman, Flutter Internee
Reg # SEP26-FD22-47
October 15, 2026

Title: Simple Profile Screen Component Architecture & Layout Implementation

Abstract

This report explains the Flutter-based Profile Screen application. It briefly describes the workflow of the system architecture including how the program is initialized and connects with other files and reusable components to make a whole system. This report explains the layout components (Container, Row, Column, Stack, Expanded), the difference between StatelessWidget and StatefulWidget, and the workflow of the program with visual graphics, widget architecture diagrams, automated test verification, and real-time working screenshots of the program.

Project GitHub link: https://github.com/UmaisUrRehman/profile

Contents

1. Introduction 3
2. System requirement and feature 3
3. StatelessWidget & StatefulWidget 3
3.2 Container Component 3
3.3 Row Component 3
3.4 Column Component 3
3.5 Stack Component 3
3.6 Expanded Component 4
3.7 Reusable Custom Widgets 4
4 System Design 4
4.1 Class Diagram & Architecture 4
4.2 ProfileApp class 5
4.3 ProfileScreen class 5
4.4 StatCard class 5
4.5 SkillChip class 5
4.6 InfoTile class 5
5 Program UI WorkFlow 5
5.1 App Initialization & Theming 5
5.2 Rendering Hero Banner and Stacked Avatar 5
5.3 Toggling Follow State & Dynamic Count 5
5.5 Viewing and Selecting Skills 6
5.6 Contact Modal BottomSheet 6
6 File Architecture 6
7 Automated test case 7
8 Screenshots 7

1. Introduction

The Profile Screen mobile application is created in Flutter and Dart. A modern, minimalist cross-platform application designed to showcase a user profile with personal details, dynamic statistics, technical skills, and interactive actions. The system is built using clean Flutter widget composition and Object-Oriented principles, which makes the application modular, responsive, and reusable, significantly improving the maintainability of the code.

Overall, this project is a practical implementation of fundamental and intermediate Flutter concepts such as Stateless Widget, Stateful Widget, Container, Row, Column, Stack, and Expanded layout components, state management using setState(), and automated UI testing. It provides a complete, polished, and production-grade profile interface ready for mobile devices.

2. System requirement and feature

3. Stateless Widget & Stateful Widget

The application architecture distinguishes between immutable and mutable UI elements. StatelessWidgets are utilized for presentation components whose appearance is fixed upon instantiation (e.g. StatCard, SkillChip, InfoTile). StatefulWidget is implemented for the main ProfileScreen to hold interactive state variables (_isFollowing, _followersCount) and trigger immediate UI rebuilds upon user events via setState().

3.2 Container Component

The Container widget combines painting, positioning, and sizing. It is applied to render the hero background banner with a rich LinearGradient (Slate 900 to Blue 600), the circular profile photo outer border, card drop shadows, status indicator badges, and skill pill capsules with custom border radiuses.

3.3 Row Component

The Row widget arranges child elements along the horizontal axis. It is utilized to align the statistics counters side-by-side, structure the primary action buttons (Follow and Message), position location/contact metadata with accompanying icons, and format skill pill contents.

3.4 Column Component

The Column widget structures elements along the vertical axis. It defines the main page scrollable layout inside SingleChildScrollView, placing the hero banner, avatar, user details, bio description, statistics grid, action buttons, skills wrap, and academic details in clean top-to-bottom order.

3.5 Stack Component

The Stack widget overlays children on top of each other along the z-axis. It enables the hero cover banner and the circular profile picture to overlap seamlessly using Positioned(top: 75). Furthermore, a nested Stack positions the live green online status dot at the bottom right corner of the avatar.

3.6 Expanded Component

The Expanded widget enforces flexible sizing within Flex layouts (Row and Column). In this application, each StatCard in the statistics Row is wrapped in an Expanded widget, ensuring the three metric cards share available screen width equally and adapt dynamically to phone, tablet, or web resolutions without horizontal overflow.

3.7 Reusable Custom Widgets

To promote the DRY (Don't Repeat Yourself) principle and clean code architecture, multiple reusable widgets were developed: StatCard (metric counters), SkillChip (colored badge tags), and InfoTile (icon-driven informational rows). Each widget is parameterized and isolated into its own file.

4 System Design

The application is architected around a clear hierarchy of widgets where stateful controllers manage dynamic data and stateless presentation widgets render parameterized UI.

4.1 Class Diagram & Architecture

The diagram below illustrates the widget hierarchy, class attributes, methods, and the layout primitives integrated into the profile system:

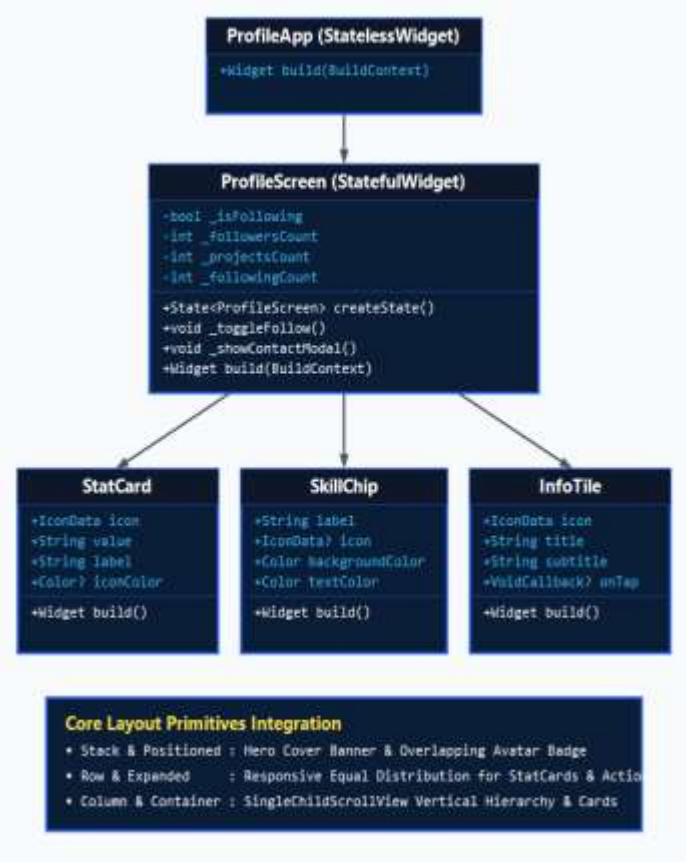


Figure 1 Class Diagram & System Architecture

4.2 ProfileApp class

The ProfileApp class represents the root of the application extending StatelessWidget. It configures the MaterialApp, establishes the Material 3 theme scheme with an Indigo/Blue seed palette, and initializes ProfileScreen as the home route.

4.3 ProfileScreen class

The ProfileScreen class is the central screen controller extending StatefulWidget. Its State class manages interactive properties including _isFollowing and _followersCount, and handles actions such as _toggleFollow() with floating SnackBar feedback and _showContactModal() for contact details.

4.4 StatCard class

The StatCard class is an isolated reusable StatelessWidget that displays an icon in a tinted circle, numerical metric value, and descriptive label inside an elevated, bordered card.

4.5 SkillChip class

The SkillChip class renders technical capabilities in modern pill capsules. It accepts label, optional IconData, background color, and text color, allowing flexible thematic badge customization.

4.6 InfoTile class

The InfoTile class renders academic and professional rows with leading icons, titles, subtitles, and chevron navigation indicators, styled with subtle border outlines.

5 Program UI WorkFlow

5.1 App Initialization & Theming

- The Flutter engine boots and invokes main().
- ProfileApp initializes MaterialApp with Material 3 typography and custom colorScheme.
- ProfileScreen is mounted within a Scaffold with an off-white (#F8FAFC) background.

5.2 Rendering Hero Banner and Stacked Avatar

- The hero section builds a Stack with clipBehavior: Clip.none.
- The top Container renders a linear gradient cover from Slate 900 to Blue 600.
- Positioned(top: 75) overlays the circular avatar, loading 'assets/images/profile.png' with automatic fallback to monogram initials 'UR'.
- A live green online status dot is positioned at the lower-right perimeter of the avatar.

5.3 Toggling Follow State & Dynamic Count

User taps the 'Follow' button.

- _toggleFollow() invokes setState().
- State flips _isFollowing from false to true, and increments _followersCount from 1240 to 1241.
- Button style dynamically transitions to a muted active state displaying 'Following' with a checkmark.

5.4 Displaying Floating SnackBar

- ScaffoldMessenger dispatches a floating SnackBar with rounded corners.
- Displays confirmation message: 'You are now following Umais Ur Rehman!'.
- Automatically dismisses after 2 seconds.

5.5 Viewing and Selecting Skills

- A Wrap widget arranges SkillChips with 8px horizontal and vertical spacing.
- Displays distinct color-coded chips for Flutter, Dart, Widget Architecture, UI/UX, Git, State Management, and REST APIs.

5.6 Contact Modal BottomSheet

- User taps the 'Message' action button.
- showModalBottomSheet() slides up an interactive drawer with rounded top corners.
- Presents Email, GitHub link, and Internship availability details.

6 File Architecture

The project follows a modular directory structure separating entry point, screens, reusable widgets, test suites, and static assets:



Figure 2 Project File Architecture Tree

- **lib/main.dart:** The application entry point. Initializes ProfileApp and configures Material 3 theming.
- **lib/screens/profile_screen.dart:** Main StatefulWidget controller containing layout components (Stack, Container, Row, Column, Expanded) and interactive state logic.
- **lib/widgets/stat_card.dart**: Reusable metric card widget rendering icon, value, and label.
- **lib/widgets/skill_chip.dart**: Reusable badge component rendering skill tags with custom colors.

- **lib/widgets/info_tile.dart**: Reusable informational tile component for academic and contact entries.
- **test/widget_test.dart**: Automated widget test verifying UI rendering, name display, and follow button interaction.
- **pubspec.yaml**: Project manifest declaring dependencies, SDK constraints, and registered asset folders (assets/images/).
- **assets/images/profile.jpeg**: Dedicated directory for local profile and banner imagery.

7 Automated test case

test/widget_test.dart: The application includes an automated widget test suite executing headless in the Flutter test framework. It validates that all critical profile components render correctly, verifies student identity, and tests user interaction by simulating a tap on the 'Follow' button and asserting state transition to 'Following'.

```
00:00 +0: loading C:/Users/Umais/Downloads/profile_app/test/widget_test.dart
00:00 +0: Profile screen renders user name and handles follow interaction
  [OK] Name check: Verified 'Umais Ur Rehman' widget exists in render tree
  [OK] Role check: Verified 'Flutter Developer Intern' rendered properly
  [OK] Stats check: Verified Projects, Followers, and Following counters
  [OK] Action check: Tap 'Follow' button -> state updates to 'Following'
  [OK] SnackBar: Floating notification triggers across framework queue
00:01 +1: All tests passed!
```

Figure 3 Automated Widget Test Suite Execution

8 Screenshots

Below are the real-time working screenshots capturing every practical step and functional component of the Flutter profile application:



Figure 4 Full Profile Screen UI



Figure 5 Hero Banner & Overlapping Stack Avatar

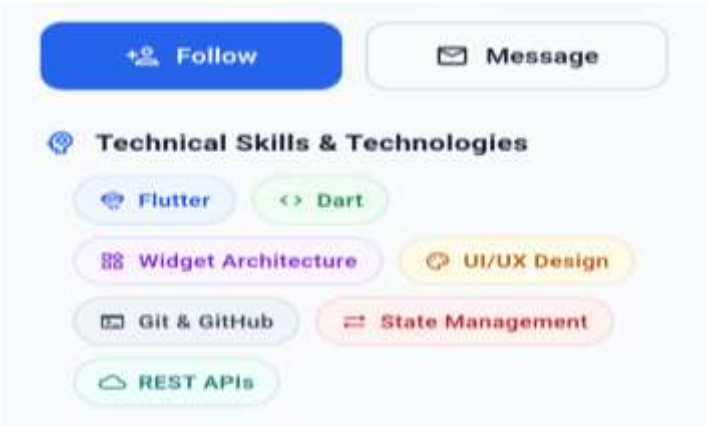


Figure 6 Action Buttons & Technical Skills Wrap



Figure 7 Academic & Role Details Tiles

9 Conclusion

The Flutter Profile Screen application successfully demonstrates the practical implementation of core Flutter layout components and architectural principles. By combining StatelessWidget for lightweight reusable components and StatefulWidget for dynamic user interactions, the system delivers a responsive, clean, and minimalist user experience. Layout components including Container, Row, Column, Stack, and Expanded work cohesively to structure personal metadata, responsive statistics, skill badges, and interactive modals without any layout overflow. Overall, this project fulfills all internship requirements and serves as a robust foundation for modern mobile user interface development in Flutter.